

= Analysis of Recursive algorithm.

Topic:

Recurrence relation

Definition:

=> A recurrence is an equation or inequality that describes a function in terms of its value on smaller inputs.

=> Recurrences are generally used in divide - and - Conquer paradigm.

- To solve a recurrence Relation means obtain a function defined on the natural numbers that satisfy the recurrence.

Example: for example the worst case running time $T(n)$ of the Merge-sort is described by the recurrence:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n=1 \\ 2T(n/2) + \Theta(n) & \text{if } n>1 \end{cases}$$

Example:

Recurrence relation of fibonacci series.

Factorial (n)

if $n == 1$ } $\rightarrow (1)$
return 1

else

return $n * \text{Factorial}(n-1)$ }

$\downarrow T(n-1)$

$$T(n) = T(n-1)$$

$$T(1) = \Theta(1)$$

$$T(n) = T(n-1) + c$$

$$T(n) = \begin{cases} 1 & n=0 \\ T(n-1) + c & \text{if } n > 1 \end{cases}$$

Example 2:

Fun (n)

if ($n > 0$) {

for ($i=0; i < n; i++$) {
Print n

fun (n-1) } $T(n-1)$

$$T(n) = T(n-1) + n + n + 1$$
$$T(n-1) + 2n + 2$$

$$= T(n-1) + n$$

$$T(n) = \begin{cases} 1 & \text{if } n=0 \\ T(n-1) + n & \text{if } n > 0 \end{cases}$$

Example 3:

= Recurrence relation of fibonacci series.

FS (n)

if $n \leq 1$ } — 1
return 1

else

return $FS(n-1) + FS(n-2)$

↳ $T(n-1) + T(n-2) + \textcircled{1}$

for addition

$$T(n) = T(n-1) + T(n-2) + 1 + 1$$

$$= T(n-1) + T(n-2) + 2$$

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ T(n-1) + T(n-2) + c & \text{if } n > 1 \end{cases}$$

Recurrence relation of Binary Search algorithm.

Binarysearch (A, f, l, e)

mid = $(f+l)/2$ — 1

if (A[mid] == e) } — 1
return mid

else

if (A[mid] > e) } — 1

Binarysearch (A, f, mid-1, e)

else

Binarysearch (A, mid+1, l, e) =

$$T(n) = T(n/2) + 4$$

$$T(n) = T(n/2) + c$$

* Recurrence relation of merge Sort.

Mergesort (lb, ub)

if (lb < ub)

(1) — mid = $(lb+ub)/2$

$T(n/2)$ — Mergesort (lb, mid)

$T(n/2)$ — mergesort (mid+1, ub)

n — Merge (lb, mid, ub)

$$= T(n/2) + T(n/2) + n + 1$$

$$= 2T(n/2) + n$$

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ 2T(n/2) + n & \text{if } n>1 \end{cases}$$

Solving recurrence relation

$\Rightarrow$ There are three methods to solve recurrence relation.

- 1) Substitution method
- 2) Recursion tree method
- 3) Master theorem

Substitution method:

$\Rightarrow$ The substitution method is a condensed way of proving an asymptotic bound on a recurrence by induction.

$\rightarrow$ In this substitution method, instead of trying to find an exact closed-form solution, we only try to find a

closed-form bound on the recurrence.
 $\Rightarrow$ There are two types of substitution methods

- o forward substitution method
- o Backward substitution method

Forward Substitution method:

- o Guess the solution.
 - o Use the mathematical induction to find the boundary condition and shows that the guess is correct.
- $\Rightarrow$ we put $n = 0, 1, 2, \dots, n$ in the recurrence relation until we see a pattern.

Backward Substitution method:

$\Rightarrow$ we put $n = n, n-1, n-2, \dots, 0$ or $n = n, n/2, \frac{n}{4}, \dots$ until we see the pattern.

Solve using forward substitution method.

$$T(n) = \begin{cases} T(1) = 1 \\ T(n) = T(n-1) + 1 \end{cases}$$

$$T(n) = T(n-1) + 1 \quad \text{--- (i)}$$

Put $n=2$

$$T(2) = T(2-1) + 1 = T(1) + 1 = 1 + 1 = 2$$

Put $n=3$

$$T(3) = T(3-1) + 1 = T(2) + 1 = 2 + 1 = 3$$

⋮

$$T(n) = n$$

Put $n=n+1$

$$\begin{aligned} T(n+1) &= T(n+1-1) + 1 \\ &= T(n) + 1 \\ &= n + 1 \end{aligned}$$

o Solve using backward substitution method.

$$T(n) = \begin{cases} T(1) = 1 \\ T(n) = n * T(n-1) \end{cases}$$

$$T(n) = n * T(n-1) \quad \text{--- (i)}$$

Put $n=n-1$

$$T(n-1) = (n-1) * T(n-2) \text{ --- (ii)}$$

Put (ii) in (i),

$$T(n) = n * (n-1) * T(n-2) \text{ --- (iii)}$$

Put $n=n-2$

$$T(n-2) = (n-2) * T(n-3) \text{ --- (iv)}$$

Put (iv) in (iii)

$$T(n-2) = n * (n-1) * (n-2) * T(n-3)$$

$\vdots$

$$T(n) = n * (n-1) * (n-2) * (n-3) \dots T(n-(n-1))$$

$$= n * (n-1) * (n-2) * (n-3) \dots T(1)$$

$$= n * (n-1) * (n-2) * (n-3) \dots * 1$$

$$= n * n \left(1 - \frac{1}{n}\right) * n \left(1 - \frac{2}{n}\right) * n \left(1 - \frac{3}{n}\right) \dots n \left(\frac{1}{n}\right)$$

$$= n^n$$

$$\in O(n^n)$$

Solve using backward substitution method.

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ T\left(\frac{n}{2}\right) + 1 & \text{if } n \geq 2 \end{cases}$$

$$T(n) = T\left(\frac{n}{2}\right) + 1 \quad \text{--- (i)}$$

Put $n = n/2$

$$T(n/2) = T(n/2^2) + 1 \quad \text{--- (ii)}$$

Put (ii) in (i)

$$T(n) = T\left(\frac{n}{2^2}\right) + 1 + 1 = T(n/2^2) + 2 \quad \text{--- (iii)}$$

Put $n = n/4$

$$T(n/4) = T(n/2^3) + 1 \quad \text{--- (iv)}$$

Put (iv) in (iii)

$$T(n) = T(n/2^3) + 3 \quad \text{--- (v)}$$

$$T(n) = T\left(\frac{n}{2^k}\right) + k \quad \text{--- (A)}$$

Let $\frac{n}{2^k} = 1$

$$n = 2^k$$

$$\log n = \log 2^k$$

$$\log n = k \quad \text{put in (A)}$$

$$T(n) = T\left(\frac{n}{2^{\log n}}\right) + \log n$$

$$= T\left(\frac{n}{n}\right) + \log n$$

$$= T(1) + \log n$$

$$= 1 + \log n$$

$$= \log n + 1 \in O(\log n)$$

Solve using backward substitution method.

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ 2T\left(\frac{n}{2}\right) + n & \text{if } n>1 \end{cases}$$

$$T(n) = 2T(n/2) + n \quad \text{--- (i)}$$

Put $n = n/2$

$$T(n/2) = 2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \quad \text{--- (ii)}$$

Put (ii) in (i),

$$\begin{aligned} T(n) &= 2 \left[2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \right] + n \\ &= 2^2 T\left(\frac{n}{2^2}\right) + n + n \end{aligned}$$

$$T(n) = 2^2 T(n/2^2) + 2n \quad \text{--- (iii)}$$

$$\text{Put } n = \frac{n}{4}$$

$$T(n/4) = 2T\left(\frac{n}{2^3}\right) + \frac{n}{4} \quad \text{--- (iv)}$$

Put (iv) in (iii)

$$T(n) = 2^3 \left(2T\left(\frac{n}{2^3}\right) + \frac{n}{4} \right) + 2n$$

$$= 2^3 T\left(\frac{n}{2^3}\right) + 3n$$

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + \frac{n}{k} \quad \text{--- (A)}$$

$$\text{let } \frac{n}{2^k} = 1$$

$$n = 2^k \quad \text{Take log b/s}$$

$$\log n = \log(2^k)$$

$$\log n = k \quad \text{put in (A)}$$

$$T(n) = 2^{\log n} T\left(\frac{n}{2^{\log n}}\right) + n \log n$$

$$= n T(1) + n \log n$$

$$= n \cdot 1 + n \log n$$

$$= n + n \log n$$

$$\in \underline{O(n \log n)}$$

Solve Using backward substitution method.

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ T(n-1) + \log n & \text{if } n>1 \end{cases}$$

$$T(n) = T(n-1) + \log(n) \quad \text{--- (i)}$$

Put $n = n-1$

$$T(n-1) = T(n-2) + \log(n-1) \quad \text{--- (ii)}$$

Put (ii) in (i)

$$T(n) = T(n-2) + \log(n-1) + \log(n) \quad \text{--- (iii)}$$

Put $n = n-2$

$$T(n-2) = T(n-3) + \log(n-2) \quad \text{--- (iv)}$$

Put (iv) in (iii)

$$T(n) = T(n-3) + \log(n-2) + \log(n-1) + \log(n) \quad \text{--- (v)}$$

⋮

$$T(n) = T(n-k) + \log(n-(k-1)) + \log(n-(k-2)) + \dots + \log(n)$$

let $n-k=1$

$$k = n-1$$

$$T(n) = T(1) + \log 2 + \log 3 + \dots + \log n$$

$$= 1 + \log(1 \cdot 2 \cdot 3 \dots n)$$

$$= 1 + \log(n!)$$

$$= 1 + \log n^n \in \Theta(n \log n)$$

Solve using backward substitution method.

$$T(n) = \begin{cases} 1 & n=1 \\ 8T\left(\frac{n}{2}\right) + n^2 & n>1 \end{cases}$$

$$T(n) = 8T\left(\frac{n}{2}\right) + n^2 \quad \text{--- (i)}$$

$$\text{Put } n = \frac{n}{2}$$

$$T\left(\frac{n}{2}\right) = 8T\left(\frac{n}{2^2}\right) + \left(\frac{n}{2}\right)^2 \quad \text{--- (ii)}$$

$$\text{Put } n = \frac{n}{4}$$
$$T\left(\frac{n}{2^2}\right) = 8T\left(\frac{n}{2^3}\right) + \left(\frac{n}{2^2}\right)^2 \quad \text{--- (iii)}$$

Put (ii) in (i)

$$T(n) = 8 \left[8T\left(\frac{n}{2^2}\right) + \left(\frac{n}{2}\right)^2 \right] + n^2$$
$$= 8^2 T\left(\frac{n}{2^2}\right) + 8\left(\frac{n}{2}\right)^2 + n^2 \quad \text{--- (iv)}$$

Put (iii) in (iv)

$$T(n) = 8^2 \left[8T\left(\frac{n}{2^3}\right) + \left(\frac{n}{2^2}\right)^2 \right] + 8\left(\frac{n}{2}\right)^2 + n^2$$

$$T(n) = 8^3 T\left(\frac{n}{2^3}\right) + 8^2 \left(\frac{n}{2^2}\right)^2 + 8 \left(\frac{n}{2}\right)^2 + n^2$$

$$= 8^3 T\left(\frac{n}{2^3}\right) + 4n^2 + 2n^2 + n^2$$

$$T(n) = 8^k T\left(\frac{n}{2^k}\right) + 2^{k-1}n^2 + 2^{k-2}n^2 + 2^{k-3}n^2 + \dots + 2^1n^2 + 2^0n^2$$

$$\text{Let } \frac{n}{2^k} = 1 \Rightarrow n = 2^k$$

$$\therefore 8^k = (2^3)^k = (2^k)^3 \quad \log n = k$$

$$T(n) = (2^k)^3 T\left(\frac{n}{2^k}\right) + 4n^2 + 2n^2 + n^2$$

$$= n^3 T\left(\frac{n}{2^{\log n}}\right) + n^2 [2^0 + 2^1 + 2^2 + \dots + 2^{k-1}]$$

$$= n^3 T(1) + n^2 \left[\frac{1(2^k - 1)}{2 - 1} \right] \quad a = 1$$

$$\therefore S = \frac{a(r^k - 1)}{r - 1} \quad r = 2$$

$$= n^3 + n^2(2^k - 1)$$

$$= n^3 + n^2(n - 1)$$

$$= n^3 + n^3 - n^2$$

$$= 2n^3 - n^2 \in O(n^3)$$

Solve using backward substitution method.

$$T(n) = \begin{cases} 1 & n=1 \\ 2T(n-1)+n & n>1 \end{cases}$$

$$T(n) = 2T(n-1) + n \quad \text{--- (i)}$$

Put $n=n-1$ in (i)

$$T(n-1) = 2T(n-2) + (n-1) \quad \text{--- (ii)}$$

$$T(n-2) = 2T(n-3) + (n-2) \quad \text{--- (iii)}$$

Put (ii) in (i)

$$T(n) = 2[2T(n-2) + (n-1)]$$

$$T(n) = 2^2 T(n-2) + 2(n-1) \quad \text{--- (iv)}$$

Put (iii) in (iv)

$$\begin{aligned} T(n) &= 2^2 [2T(n-3) + (n-2)] + 2(n-1) \\ &= 2^3 T(n-3) + 2^2(n-2) + 2(n-1) \quad \text{--- (v)} \end{aligned}$$

Put $n=n-3$

$$T(n-4) = 2T(n-4) + (n-3) \quad \text{--- (vi)}$$

Put (vi) in (v)

$$= 2^3 [T(n-4) + (n-3)] + 2^2(n-2) + 2(n-1)$$

$$= 2^4 T(n-4) + 2^3(n-3) + 2^2(n-2) + 2^1(n-1)$$

$$= 2^4 T(n-4) + 2[4n-12 + 2n-4 + n-1]$$

$$= 2^4 T(n-4) + 2[7n-17]$$

$$T(n) = 2^k T(n-k) + (2^k - 1)n - c$$

Let

$$n-k=1$$

$$k=n-1$$

$$T(n) = 2^{n-1} T(n-(n-1)) + (2^{n-1} - 1)n - c$$

$$= 2^{n-1} T(1) + (2^{n-1} - 1)n - c$$

$$= 2^{n-1} + (2^{n-1} - 1)n - c$$

$$= 2^{n-1} + n2^{n-1} - n - c$$

$$= 2^{n-1} + n2^{n-1} - n - c$$

$$= 2^{n-1} + (2^{n-1} - 1)n - c$$

$$\in O(2^n)$$